

23CSE455 Design Patterns

Design Pattern Case Study Report

Airline Reservation Application using NestJS

Design Pattern Analysis in a NestJS-Based Airline Reservation Application

Team: A8
Application: NestJS Airplane Reservation System
Framework: NestJS + GraphQL + TypeORM
Baseline commit: ae2c24869377ce2f7a29022cd237d808338156ea
Report date: 2026-10-06

Students

Roll number	Student name	Batch
AM.SC.U4CSE23016	Balashankar Mohan	CSE-A
AM.SC.U4CSE23125	Josewin Anto J	CSE-B
AM.SC.U4CSE23128	Karthik Kishor	CSE-B
AM.SC.U4CSE23141	Pavithra Nair	CSE-B

1 Application and Architecture

1.1 Application overview

The case study is a GraphQL-based airline reservation application built using NestJS. The provided README includes information on user management, flight management, booking, seat management, session authentication, Google OAuth, role-based access control, query batching with DataLoader, and background processing with BullMQ. The baseline repository does not have a payment entity, payment service, payment resolver, or payment controller, so payment is considered an unimplemented capability in the baseline, not a confirmed pattern instance.

The application is the open-source NestJS Airplane Reservation System, originally published at <https://github.com/zeyadAlbadawy/Airport-Reservation-System.git>. The supplied baseline is identified by commit `ae2c24869377ce2f7a29022cd237d808338156ea`, and its unmodified snapshot is stored under `data/code/`. The team's working repository, https://github.com/DevBM04/DP_Case_study, holds the baseline snapshot together with the Part 2 evolution commits (Change 1 `b731f57`, Change 2 `febb4c4`, Change 3 `34e8621`). The report treats the commit SHAs and the saved source snapshots as the authoritative version identifiers.

1.2 Environment

Component	Version / evidence
Node.js	v25.9.0 for the baseline verification record; Part 2 change tests used v24.15.0
npm	11.12.1
TypeScript	5.9.3 / package range <code>~5.7.3</code>
NestJS Core	<code>~11.0.1</code> baseline
NestJS GraphQL	<code>~13.2.0</code> ; Apollo Server <code>~5.1.0</code>
TypeORM	<code>~0.3.27</code>
DataLoader	<code>~2.2.3</code>
BullMQ	<code>~5.65.1</code> ; <code>@nestjs/bullmq</code> <code>~11.0.4</code>
PostgreSQL driver	<code>pg</code> <code>~8.16.3</code>

1.3 Scope

The inspected application scope includes `src/flight`, `src/booking`, `src/seat`, and `src/users`, with supporting application configuration in `src/app.module.ts`. The pattern analysis does not cover framework internals, like the implementation of Passport's Google Strategy or TypeORM standard repository mechanics, as these are more of a dependency/framework collaboration rather than an application-level one. The analysis also rejects the `allAuthGuard` structure as a formal Composite pattern based on the supplied Part 1 verification.

1.4 Architecture and request flow

In NestJS, GraphQL queries and mutations are exposed with resolvers. Domain operations services TypeORM repositories for persisting entities DataLoader providers for batching relation lookups BullMQ for decoupling email jobs from synchronous request processing Creates role-based access. The `rolesRestrict` factory.

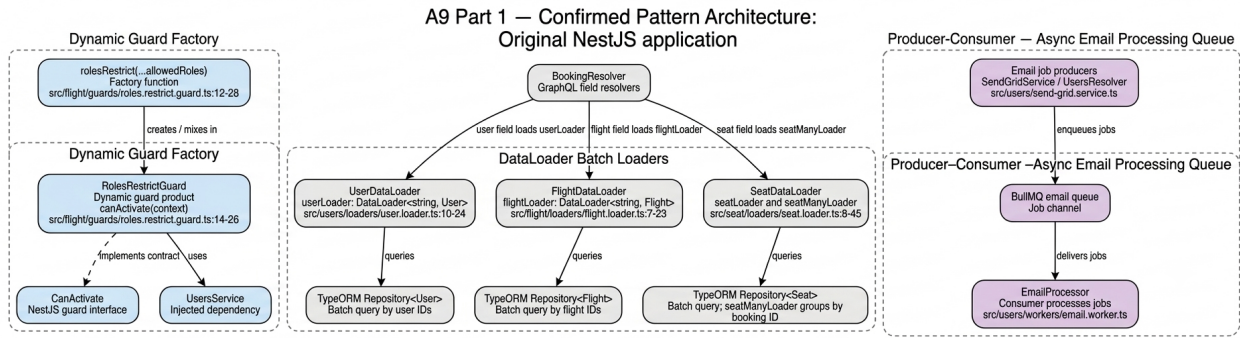


Figure 1: Baseline pattern architecture supplied with the case-study evidence.

A representative booking request follows this flow: a GraphQL mutation enters `BookingResolver`, the resolver delegates booking and seat creation to services. Relation fields are resolved by request-scoped ‘DataLoaders’. An email payload is handed off to an operation such as booking confirmation or flight cancellation that produces a notification. `SendGridService`; the service enqueues a job on the BullMQ email queue and `EmailProcessor` consumes it asynchronously.

1.5 Run and verification instructions

The baseline evidence records the following results:

- `npm run build`: passed, exit code 0.
- `npm test`: passed; 10 test suites and 27 tests passed in the preserved baseline verification record.
- `npm run test:e2e`: failed, exit code 1, because the supplied e2e Jest configuration lacked the required `moduleNameMapper` mapping for `src/` absolute imports.

The Part 2 change snapshots were tested with `npm test`; Change 1 passed 14/14, Change 2 passed 21/21, and Change 3 passed 27/27 tests.

2 Part 1 – Pattern Discovery

2.1 Method

Pattern candidates were identified by repository archaeology and was checked against actual source collaborations. A candidate was retained only when its roles could be mapped to real functions, classes, interfaces, queues, providers, or resolver methods and the interactions demonstrated the claimed intent. A class name or LLM assertion alone was not considered enough evidence. This matches the case-study instructions, which require real participant mappings and source locations for every confirmed collaboration.

2.2 Baseline totals

The baseline contains **3 distinct confirmed pattern types** and **5 independent confirmed pattern instances**. The instances are one Dynamic Guard Factory collaboration, three Batch Loader collaborations, and one Producer–Consumer collaboration. The concrete participant classes/providers represented in these collaborations are **9 distinct symbols** when shared symbols are counted once: `rolesRestrict`, `RolesRestrictGuard`, `CanActivate`, `UserDataLoader`, `FlightDataLoader`, `SeatDataLoader`,

`SendGridService`, `UsersResolver`, and `EmailProcessor`. Participants also include the BullMQ queue channel and TypeORM repository, which are framework/dependency objects and not separate application classes.

Rejected/uncertain observations include `allAuthGuard` as a pattern-like structure rather than a confirmed Composite pattern, TypeORM repository/data-mapper behavior as framework mechanism, and Passport Google Strategy as a framework mechanism. The baseline payment subsystem is not implemented.

2.3 Pattern inventory

Pattern type	Confirmed instance names	Instances	Concrete application classes/providers
Dynamic Guard Factory	Role-Based Guard Synthesis	1	2 main classes/functions + interface
Batch Loader	User Entity Batch Loading; Flight Entity Batch Loading; Seat Entity Batch Loading	3	3 loader providers
Producer-Consumer	Async Email Processing Queue	1	4 main application participants

Table 1: Baseline pattern inventory.

2.4 Confirmed pattern evidence

2.4.1 Role-Based Guard Synthesis – Dynamic Guard Factory

Category: Creational (as recorded in the supplied pattern dataset). The implementation is a NestJS-specific factory/mixin idiom.

Purpose and origin. The collaboration parameterizes the authorization process by accepting any set of allowed roles and returning the guard type of NestJS. This eliminates the need to create static guards for each role set.

Role	Actual participant	Source location
Factory function	<code>rolesRestrict(...allowedRoles)</code>	<code>src/flights/guards/roles.restrict.guard.ts:12-28</code>
Dynamic product	<code>RolesRestrictGuard</code>	same file, lines 14–27
Product interface	<code>CanActivate</code>	same file, lines 12–16
Dependency	<code>UserService</code>	same file, lines 10–15, 22
Consumers	Flight/Booking resolver guard annotations	<code>FlightResolver</code> and <code>BookingResolver</code>

The function closes over `allowedRoles`, defines a local class implementing `CanActivate`, checks the authenticated session user through `UserService`, and returns `mixin(RolesRestrictGuard)`. It is thus related to the production of dynamic authorization guards in a direct way rather than just being associated with a class called “Guard.”

2.4.2 User Entity Batch Loading

Category: Architectural. `UserDataLoader` is request-scoped and wraps `DataLoader`. Its batch function queries users using `TypeORM In(userIds)`, builds a map, and restores the original requested key order. `BookingResolver.user()` calls `userLoader.load(booking.userId)`.

Role	Participant	Source location
Batch loader provider	<code>UserDataLoader</code>	<code>src/users/loaders/user.loader.ts:10-24</code>
Loader instance	<code>userLoader</code>	<code>src/users/loaders/user.loader.ts:15-23</code>
Target repository	<code>Repository<User></code>	<code>src/users/loaders/user.loader.ts:11-18</code>
Relational resolver	<code>BookingResolver.user</code>	<code>src/booking/booking.resolver.ts:114-117</code>

This is aimed at avoiding a situation whereby N+1 queries are triggered when resolving the user relation of multiple bookings within a single execution cycle.

2.4.3 Flight Entity Batch Loading

`FlightDataLoader` follows the same batching structure for flight relations. It accepts flight IDs, executes one repository lookup with `In(flightIds)`, maps the returned flights by ID, and restores the requested order. `BookingResolver.flight()` invokes the loader.

Role	Participant	Source location
Batch loader provider	<code>FlightDataLoader</code>	<code>src/flight/loaders/flight.loader.ts:7-23</code>
Loader instance	<code>flightLoader</code>	<code>src/flight/loaders/flight.loader.ts:12-22</code>
Target repository	<code>Repository<Flight></code>	<code>src/flight/loaders/flight.loader.ts:8-15</code>
Relational resolver	<code>BookingResolver.flight</code>	<code>src/booking/booking.resolver.ts:119-122</code>

2.4.4 Seat Entity Batch Loading

`SeatDataLoader` contains two related loader instances. `seatLoader` maps individual seat IDs to `Seat` entities, while `seatManyLoader` maps booking IDs to arrays of seats. Both use request-scoped `DataLoader` batching and `TypeORM In(...)` queries.

Role	Participant	Source location
Batch loader provider	<code>SeatDataLoader</code>	<code>src/seat/loaders/seat.loader.ts:8-45</code>
Single-seat loader	<code>seatLoader</code>	lines 14–23
Booking-to-seats loader	<code>seatManyLoader</code>	lines 26–43
Target repository	<code>Repository<Seat></code>	lines 9–17, 28–30
Relational resolver	<code>BookingResolver.seat</code>	<code>src/booking/booking.resolver.ts:124-127</code>

2.4.5 Async Email Processing Queue – Producer–Consumer

Category: Architectural. The producer side consists of application services/resolvers that enqueue jobs, the BullMQ `email` queue is the channel, and `EmailProcessor` is the consumer worker.

Role	Participant	Source location
Job producer	<code>SendGridService.send</code>	<code>src/users/send-grid.service.ts:30-42</code>
Job producer	<code>UsersResolver</code>	<code>src/users/users.resolver.ts:78-81</code>
Queue channel	<code>BullMQ email queue</code>	<code>src/app.module.ts:27-35</code>
Consumer worker	<code>EmailProcessor</code>	<code>src/users/workers/email.worker.ts:8-25</code>

`SendGridService.send()` adds a `sending-token-sendGrid` job to the queue rather than performing the email operation synchronously. `EmailProcessor.process()` consumes jobs and dispatches the appropriate email operation. The collaboration therefore separates request processing from I/O-heavy background work.

2.5 Verification and rejected candidates

The supplied Part 1 analysis explicitly rejects `allAuthGuard` as a formal Composite pattern and categorizes TypeORM repositories and Passport's Google Strategy as framework constructs. This is vital since the number of collaboration at application level counted only those collaborations that had source evidence. The payment module was also validated to be missing from the baseline.

3 Part 2 – Evolution and Refactoring

The three changes that have been applied are: CH01 Flight Cancellation, CH02 Seat Class Extension, and CH03 Ticket Change. There has been mention made about a new payment provider in the assignment, however, the change artifacts provided clearly state that payment/ refund functionality is not in scope of the work performed.

3.1 CH01 – Flight Cancellation

Requirement. Instead of destructive deletion of the flight, use a non-destructive lifecycle state, do not allow new bookings for the cancelled flight, free up the relevant seat and enable cancellation by administrators or crew members.

Implementation. A `FlightStatus` enum introduces `ACTIVE` and `CANCELLED`. `FlightService.cancelFlight` persistence changes from destructive delete to status update, initializes available seats, frees up seats, and sends notification task. `BookingService.createBooking` will not accept cancelled flights. The resolver makes available the cancellation feature and maintains compatibility with deletion mutation.

Pattern effects.

- Dynamic Guard Factory: **extended**; `rolesRestrict(Role.ADMIN, Role.CREW)` protects the cancellation operation.
- Producer–Consumer: **extended**; `FlightService` becomes an additional producer for cancellation notices.
- All three Batch Loader collaborations: **preserved**; the Flight entity gains lifecycle state without changing the batching mechanism.

Test evidence. Change 1: 10 suites, 14/14 tests passed.

3.2 CH02 – Seat-Class Expansion

Requirement. Introduce `FIRST`, `BUSINESS`, and `ECONOMY`; enforce row/class allocation rules; support explicit and inferred classes; preserve seat uniqueness, availability, and `DataLoader` behavior.

Rules for the supplied domain are 1-3 = FIRST, 4-8 = BUSINESS, and 9-20 = ECONOMY. Class values explicitly assigned are verified using the above ranges, and missing class values are determined deterministically.

Pattern effects.

- Dynamic Guard Factory: **preserved**.
- Producer–Consumer: **preserved**.
- Seat Entity Batch Loading: **preserved**; the loader continues to hydrate Seat entities, now with the added `seatClass` field.
- User and Flight Batch Loading: **preserved**.

Test evidence. Change 2: 10 suites, 21/21 tests passed.

3.3 CH03 – Ticket Modification

Requirement. Use the current Booking aggregate instead of adding another duplicate Ticket entity. Facilitate seat changes, seat class changes, and flight changes while ensuring ownership, seat availability, flight availability, and cancellations.

The new `UpdateBooking` input contains `bookingId`, optional `flightId`, and optional target seats. `BookingService.modifyBooking` validates ownership, prevents modification of cancelled flights, restores capacity on the old flight when transferring, reduces capacity on the target flight, and allocates replacement seats through `SeatService`. Resolver mutations are protected with `rolesRestrict('USER')`.

Pattern effects.

- Dynamic Guard Factory: **extended**; it now guards `modifyBooking` and backward-compatible `updateBooking` operations.
- User, Flight, and Seat Batch Loaders: **preserved**; they resolve the updated booking relationships.
- Producer–Consumer: **preserved**; the existing queue remains the asynchronous messaging infrastructure.

Test evidence. Change 3: 10 suites, 27/27 tests passed.

3.4 Pattern count evolution

Snapshot	Guard Factory	Batch Loader	Producer–Consumer	Total instances
Original	1	3	1	5
Change 1	1	3	1	5
Change 2	1	3	1	5
Change 3	1	3	1	5

Table 2: Confirmed pattern counts across source snapshots.

This stability of the count is not an indicator of architectural constancy. There was stability in the collaboration roles, whereas their responsibility underwent change, with the guard factory acquiring new protected actions and the email producer role being extended in the event of cancellation of flights.

3.5 Refactoring and design impact

These modifications involve refactoring of concrete components as opposed to just addition of new methods. CH01 involves refactoring of the destructive flight deletion feature into a lifecycle aware status change. CH03 involves refactoring of the booking update input into an expressive DTO that contains booking id, optional target flight and nested seat layout information.

In the absence of any defined LOC or cyclometric-complexity measurement criteria for the application changes from the Part 2 notes provided, this report will not provide before-and-after numbers. This functional proof will depend upon the unit test results and code differences that have been documented.

4 Part 3 – Cross-Language Comparison

The two key identified patterns include Dynamic Guard Factory Pattern and Producer–Consumer Pattern. These implementations were specifically developed without the presence of NestJS, Redis, BullMQ, and networks so that the comparison is made possible on Java, Python, JavaScript, and C++.

4.1 Common specifications

For Dynamic Guard Factory, the reduced problem is to design a role-checking guard based on a set of allowed roles and check the correctness of a provided user role. Seven test cases were implemented for each language.

For Producer–Consumer, the reduced problem is a deterministic FIFO job queue with producer actions and consumer processing depending on the job type. Four test cases were implemented for each language.

4.2 Dynamic Guard Factory results

Language	LOC	Max CC	Test	Median ms	Notes
Java	19	2	7/7	78.3662	JVM startup, class loading and test execution included
Python	9	2	7/7	34.3508	Python process startup, module loading and test execution included
JavaScript	12	2	7/7	41.5423	Node process startup, module loading and test execution included
C++	18	2	7/7	7.6309	Executable startup and test execution included

Table 3: Dynamic Guard Factory cross-language measurements.

However, all four implementations passed all tests. C++ has the lowest process time recorded in the reduced benchmark, while Python has the smallest production code size. This timing is not meant to be a benchmark for performance of languages for the original NestJS application but rather for the snippets alone.

4.3 Producer–Consumer results

Language	LOC	Max CC	Test	Median ms	Notes
Java	51	4	4/4	94.4393	JVM startup, class loading and test execution included
Python	24	4	4/4	62.5046	Python process startup, module loading and test execution included
JavaScript	38	4	4/4	51.8063	Node process startup, module loading and test execution included
C++	38	4	4/4	10.8482	Executable startup and test execution included

Table 4: Producer–Consumer cross-language measurements.

4.4 Batch Loader results

Language	LOC	Max CC	Test	Median ms	Notes
Java	71	3	7/7	91.6325	JVM startup, class loading and test execution included
Python	32	3	7/7	23.8342	Python process startup, module loading and test execution included
JavaScript	48	2	7/7	41.2825	Node process startup, module loading and test execution included
C++	79	5	7/7	11.2842	Executable startup and test execution included

Table 5: Batch Loader cross-language measurements.

All four implementations passed all seven tests, validating batched repository access, unique-key handling, key ordering, missing-record handling, and caching. C++ recorded the lowest process time in this reduced benchmark, while Python had the second-lowest. These measurements include process startup and test execution and are not intended to compare language performance in the original NestJS application.

4.5 Runtime and compilation environment

Language	Runtime/compiler
Java	Java 25.0.1 LTS / javac 25.0.1
Python	Python 3.11.9
JavaScript	Node.js v22.12.0
C++	g++ (MSYS2) 14.2.0 / C++17

Compilation was done before timing; one warmup and five timings were done, with the median value presented. Compilation time was not included. Raw data were saved for each pattern/language combination. `timing_raw.csv` file.

4.6 Structural comparison

Dynamic Guard Factory naturally maps to first-class functions/closures in Python and JavaScript, but factory methods/functions and concrete guard objects in Java and C++. However, the fundamental design concept remains identical: create guards depending on permitted roles and return an object capable of conducting the validation.

Producer-Consumer pattern implementation demonstrates significant language level disparities. In Java, this problem is solved through explicit classes and queue types; in Python – using a concise object-oriented model; in JavaScript – through the object-oriented model available at run time; in C++ – through explicit types and queue support from the standard library. Nevertheless, all implementations remain faithful to the roles of producer, channel, and consumer.

4.7 Comparative dataset

The report data contains information regarding LOC source, the maximum complexity of cyclomatic function, runtime/ compiler, test command/result, median execution time, and language-related information. It is worth noting that the portability of the pattern is not affected by the implementation technique used. The execution times should only be considered within the given execution context and snippet scope.

5 LLM Records and Human Verification

The repository contains LLM-related documentation for Parts 1–3. The records distinguish preserved outputs from reconstructed or representative documentation where original transcripts were unavailable.

Part	Model / Date	LLM Record and Evaluation	Verification
Part 1	Gemini 3.6 Flash (High), Oct. 2026	<code>Part1/part1_antigravity_output.md</code> . Output was partially correct; a candidate pattern was rejected.	Pattern CSV and source inspection.
Part 2	Gemini 1.5 Pro, Oct. 6, 2026	Change prompts and outputs. CH01 was correct; the CH02 output was not recoverable; the CH03 record is reconstructed/representative.	Change test results.
Part 3	GPT-5.6 Luna, Oct. 6, 2026	<code>Part3/llm/part3_prompt.txt</code> . The implementation record was reconstructed; no literal transcript was preserved.	Cross-language tests and timing files.

Table 6: LLM records and human verification across project parts.

The team performed human verification using source-code locations, diffs in the change records, unit-test outputs, and cross-language CSV results. Missing execution times were not estimated, and no payment-provider implementation was invented.

6 Findings and Contributions

6.1 Findings

1. The baseline This application features five validated pattern combinations within three different pattern groups, namely one Dynamic Guard Factory, three Batch Loaders, and one Producer-Consumer queue.
2. The strongest architectural reuse is visible in the DataLoader collaborations: the same batching strategy is applied independently to users, flights, and seats.
3. The Dynamic Guard Factory is resilient to new authorization requirements. It is extended rather than replaced when flight cancellation and booking modification introduce new role-protected operations.

4. Flight cancellation demonstrates that a pattern can evolve because of a domain lifecycle change: the existing Producer–Consumer infrastructure gains a new producer without changing the queue/consumer contract.
5. Seat-class expansion changes the domain model without forcing changes to the DataLoader collaboration, illustrating separation between entity representation and relation-loading infrastructure.
6. The pattern counts remain constant across all three changes, but participant responsibilities and interactions change. Therefore, pattern evolution cannot be inferred from counts alone.
7. The cross-language implementations show that the same pattern intent can be expressed through different abstraction mechanisms while retaining equivalent behavior.
8. Payment integration remains a limitation of the supplied implementation. A new payment provider was not implemented, and the change requirements explicitly excluded payment gateways, refunds, and monetary transactions.

6.2 Contributions

Table 7: Team Member Contributions

Roll Number	Name	Batch	Actual Contribution
AM.SC.U4CSE23016	Balashankar Mohan	CSE-A	Analyzed the original application, identified design pattern instances, and documented Part 1 findings.
AM.SC.U4CSE23125	Josewin Anto J	CSE-B	Prepared overall documentation, conducted baseline analysis and testing, documented test results, and organized supporting evidence.
AM.SC.U4CSE23128	Karthik Kishor	CSE-B	Analyzed feature changes, documented pattern evolution, and prepared Part 2 records and diagrams.
AM.SC.U4CSE23141	Pavithra Nair	CSE-B	Implemented selected patterns in four languages, ran tests, collected timing measurements, and documented Part 3 results.

7 References and Evidence Files

1. Supplied case-study instructions: `DP_Casestudy.pdf`; defines the required CSV datasets, report sections, source evidence rules, and team-folder structure.
2. Supplied report template: `DP_Casestudy.pdf`; defines the required report organization from Application and Architecture through Findings and References.
3. Baseline application source: commit `ae2c24869377ce2f7a29022cd237d808338156ea`; source snapshot stored under `data/code/`.
4. Part 1 evidence: `Part1/patterns.csv`, `Part1/Pattern_count.csv`, `Part1/part1_antigravity_output.md`, and `Part1/pattern_architecture/`.
5. Part 2 evidence: `Part2/pattern_evolution.csv`, `Part2/Pattern_count.csv`, change requirements, diffs, test results, and evolution diagrams.
6. Part 3 evidence: `Part3/pattern_crosslanguage.csv`, four-language snippets, test runners, README files, and raw timing CSVs.

7. NestJS, DataLoader, TypeORM, and BullMQ are used by the supplied application; the report's pattern claims are based on the application's actual collaborations rather than framework documentation alone.

8 Team Folder and Dataset Checklist

The supplied A8 folder contains the Part 1, Part 2, and Part 3 supporting artifacts. Before final submission, the following items should be checked against the faculty repository instructions:

- `A8_Report.tex` and compiled `A8_Final_Report.pdf`.
- One final dataset folder containing `patterns.csv`, `pattern_counts.csv`, `cross_language.csv` (or the team's accepted filename), and `llm.csv`.
- Part 1 architecture diagram source and rendered image.
- Part 2 change requirements, diffs, test results, and updated diagrams.
- Part 3 four-language implementations, tests, READMEs, and raw timing evidence.
- README with source/version information, environment, run commands, inspected modules, and limitations.
- No passwords, dependency caches, or compiled binaries.

End of report